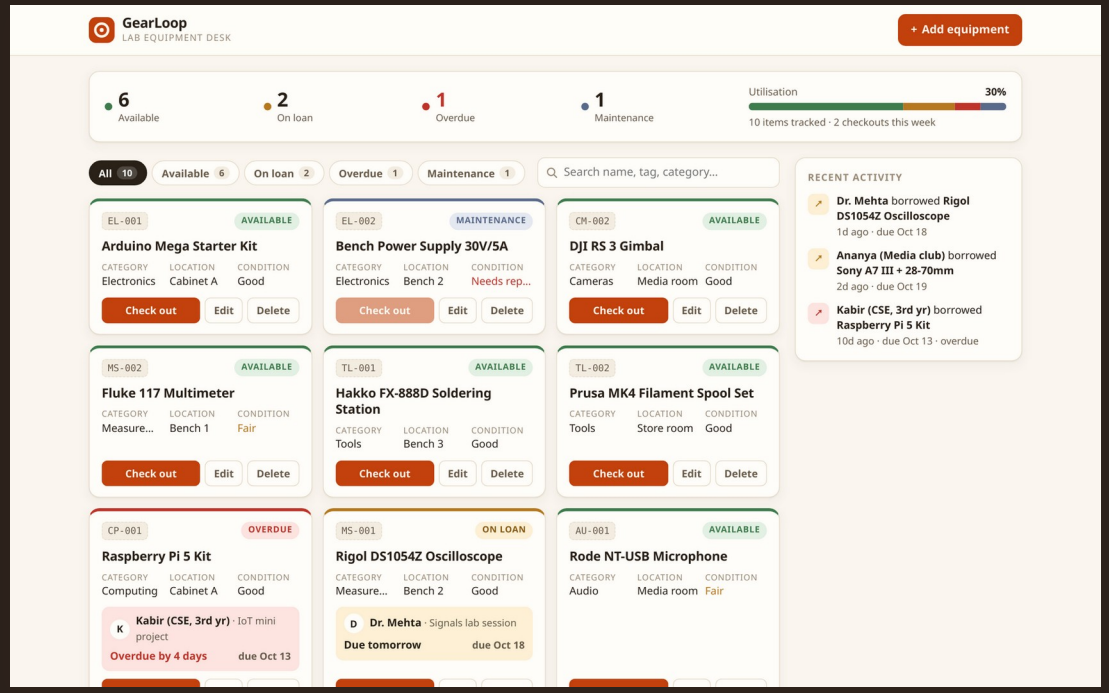
GearLoop

A lab equipment desk, taken from a laptop to a monitored Kubernetes cluster



DevOps Final Capstone · Aastik Nayyar

React · FastAPI · PostgreSQL · Docker · GitHub Actions · Trivy · GHCR · Terraform · Kubernetes · Helm · Ingress · HPA · Prometheus · Grafana

What GearLoop does

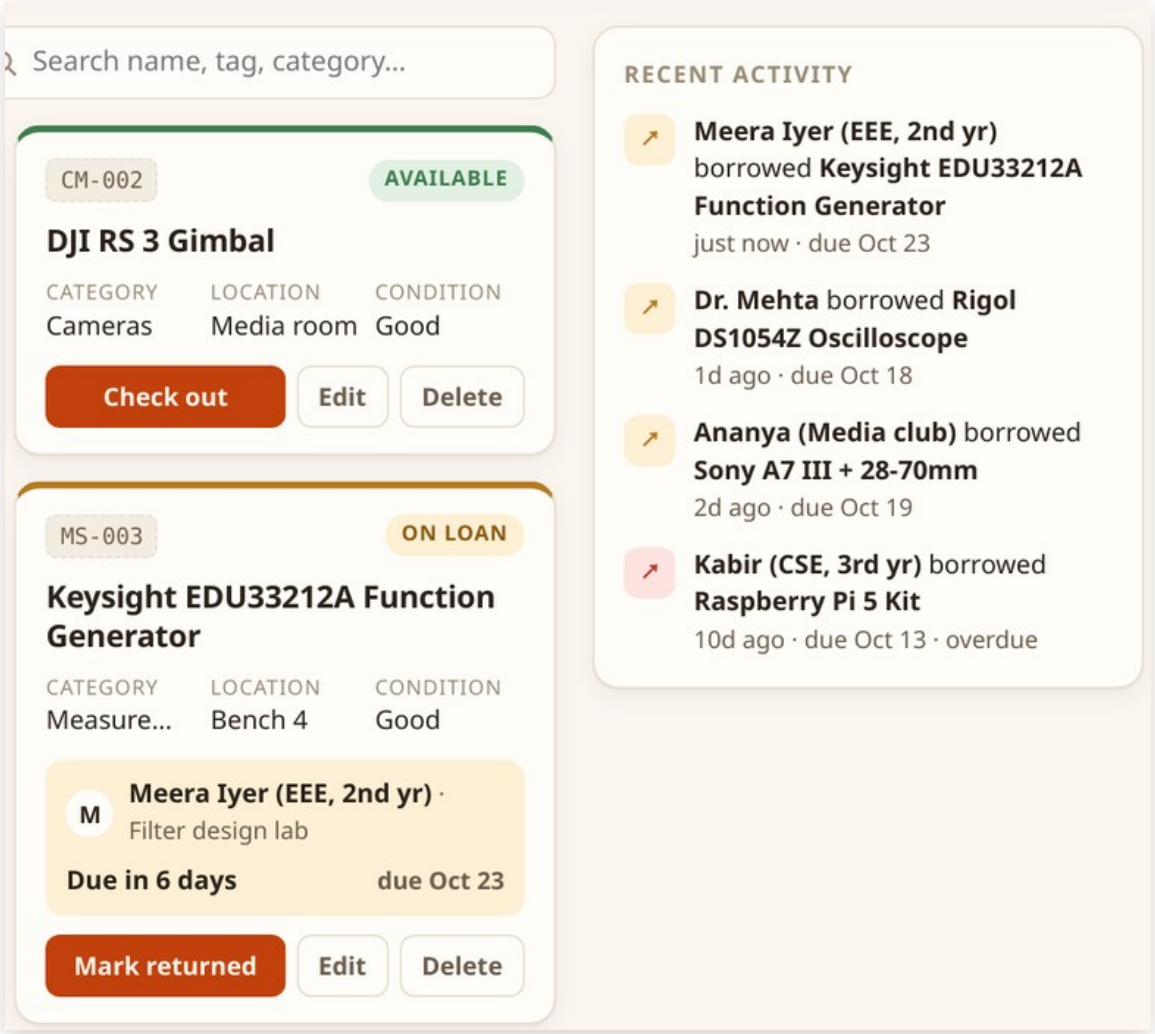
Labs lend out oscilloscopes, cameras and dev kits on paper slips. Items go missing, nobody knows what is overdue, and broken gear gets lent out again.

Register equipment
asset tag, category, location, condition

Check out and return
borrower, purpose, 1–30 day loans

Overdue at a glance
status derived from due dates

Repairs block lending
"Needs repair" moves an item to Maintenance



Tech stack

Frontend

React 19 · Vite 8 · plain CSS · vitest

Backend

FastAPI · SQLAlchemy 2 · Pydantic · Alembic

Database

PostgreSQL 17 (StatefulSet + PVC)

Containers

multi-stage Docker · Compose · GHCR

CI/CD

GitHub Actions · Trivy · Argo CD (GitOps)

Infrastructure as Code

Terraform: kubernetes · helm · random

Kubernetes

minikube · Helm · ingress-nginx · HPA

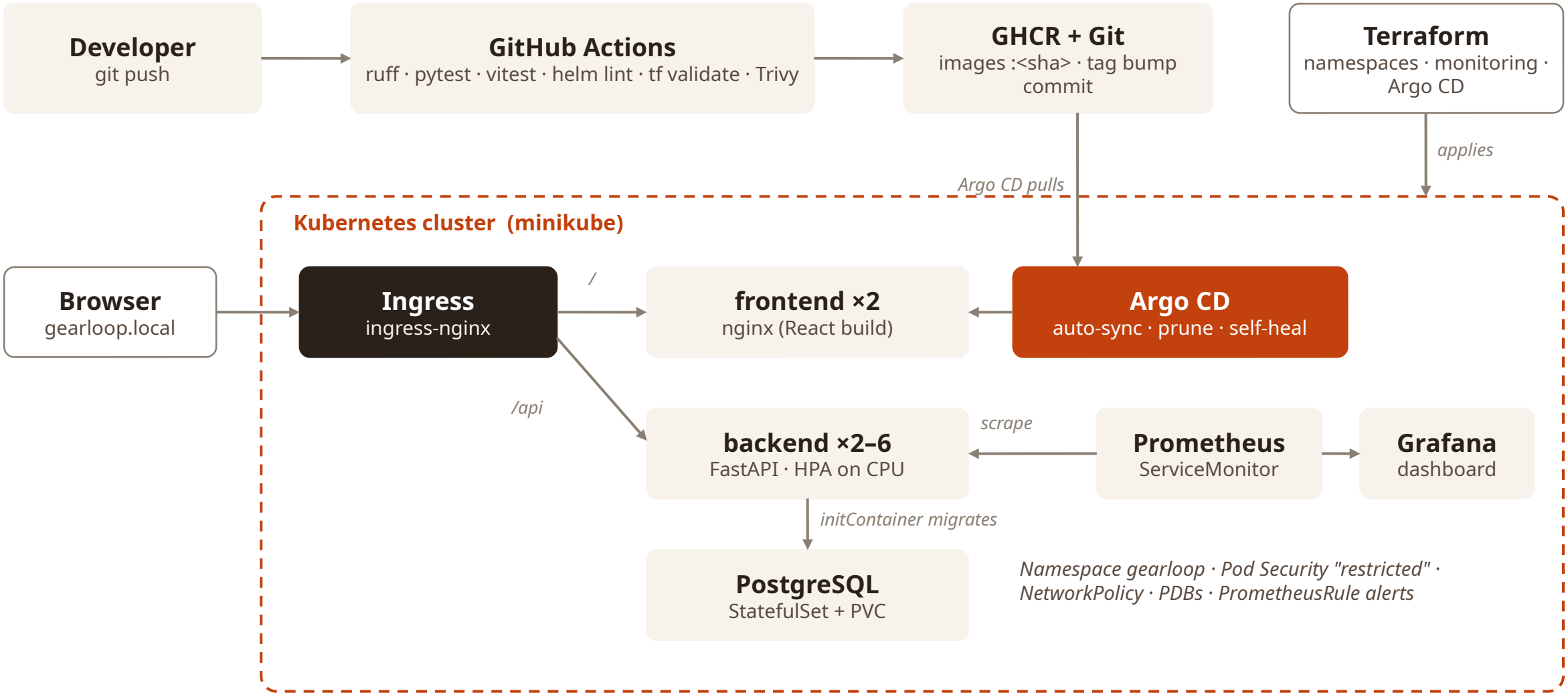
Observability

Prometheus · Grafana · Alertmanager

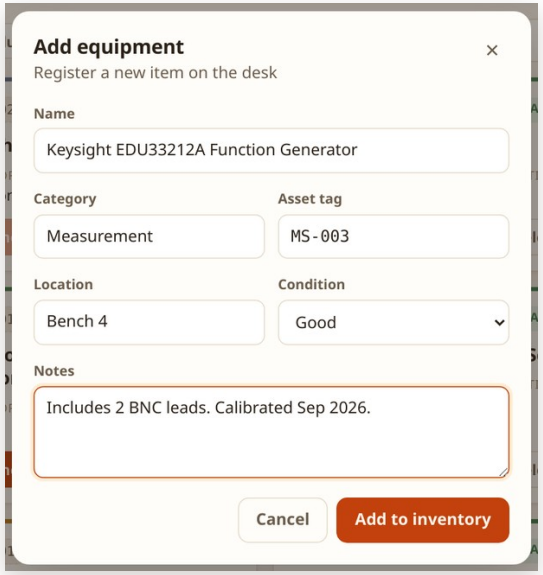
Quality gates

pytest · ruff · alembic check · actionlint

Architecture: laptop to monitored cluster

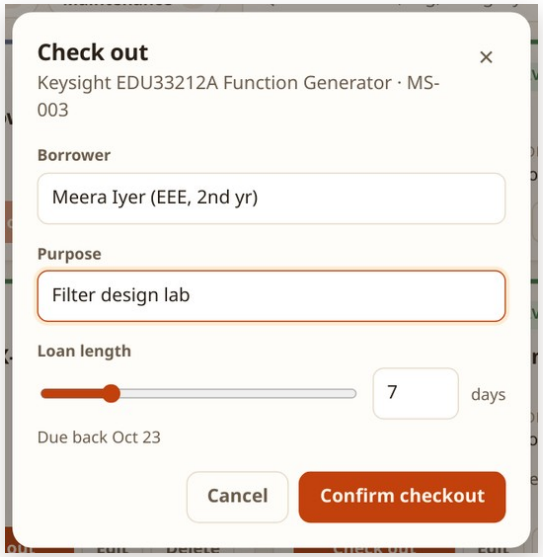


Core workflows in the UI



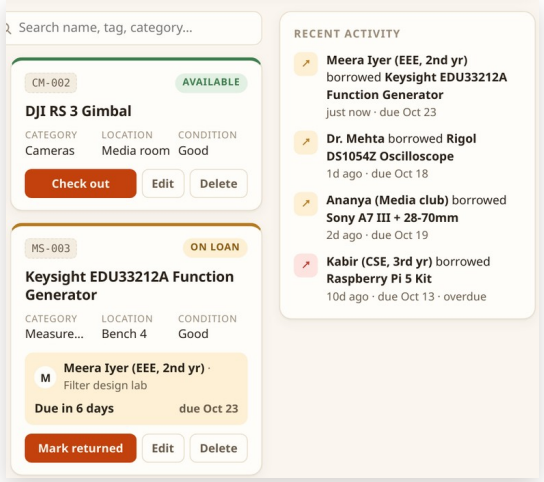
1 Register

Asset tags are unique; the API answers 409 on duplicates.



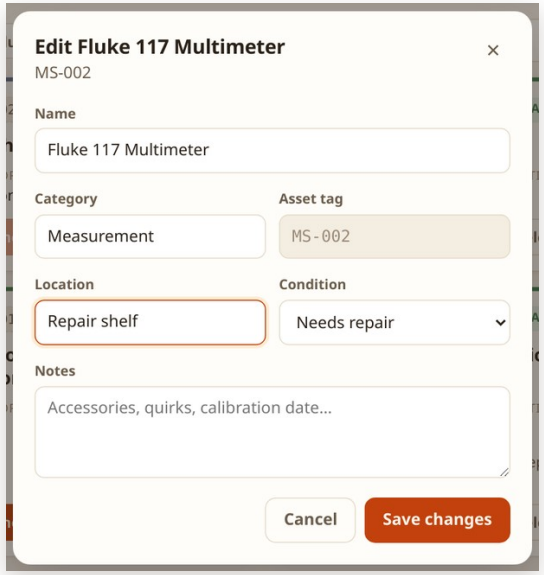
2 Check out

Borrower, purpose and a 1–30 day loan; due date shown live.



3 On loan

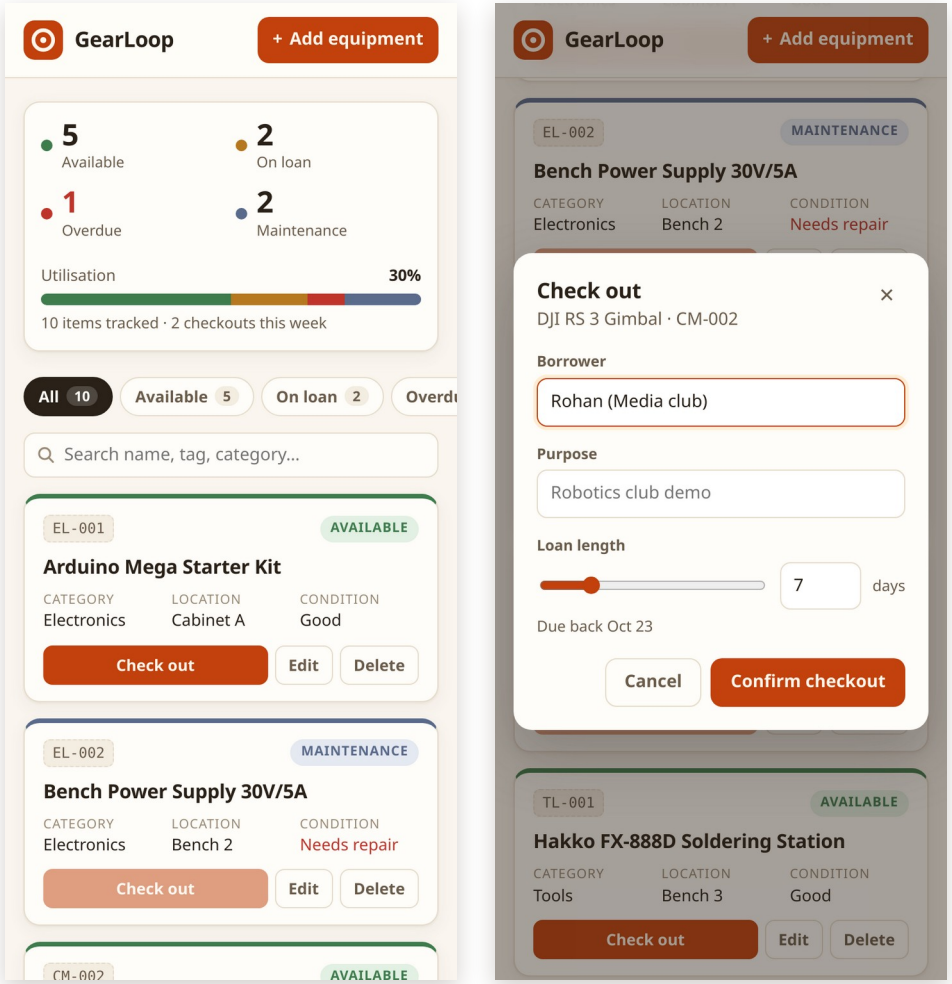
Card shows who has it and when it is due; the activity feed updates.

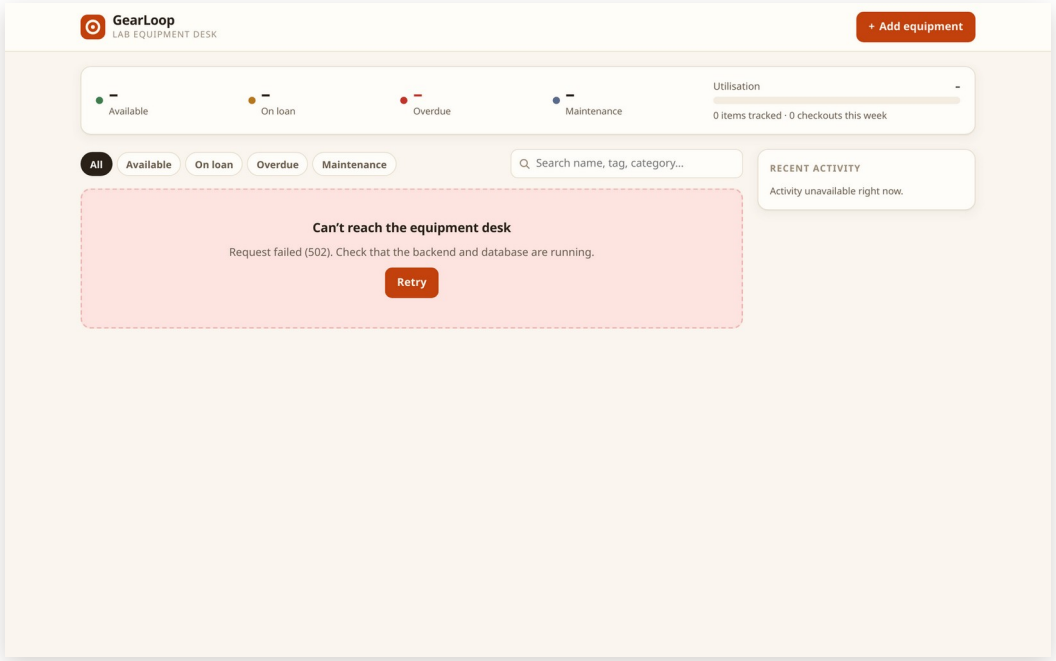


4 Repair

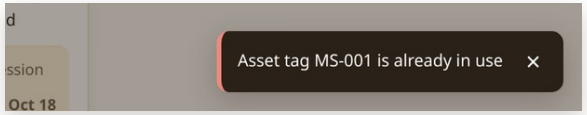
Setting "Needs repair" blocks new checkouts until fixed.

Responsive, and honest about failure





Backend container stopped: clear message + Retry instead of a blank page

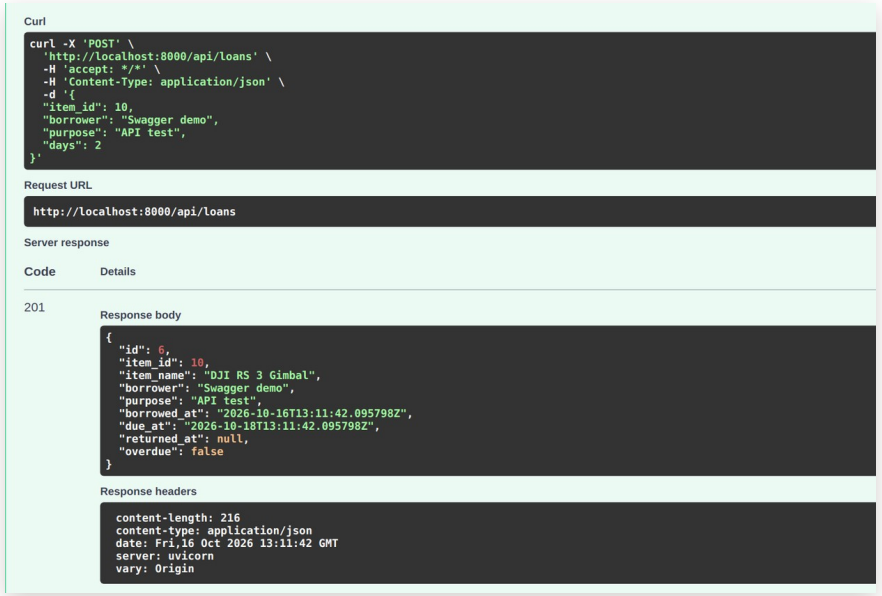


API errors (409 / 422) as toasts

- Phone layout down to 360px
- Summary stacks, chips scroll
- In-app dialogs, no browser confirm()

FastAPI backend: 10 endpoints + probes

Method	Path	Purpose
GET	`/health` `/ready` `/metrics`	liveness · readiness · metrics
GET	`/api/stats`	counts, utilisation %, checkouts this week
GET	`/api/items?status=&q=`	list, filter, search
POST	`/api/items`	register (409 on duplicate tag)
PUT · DELETE	`/api/items/{id}`	update · delete (409 while on loan)
GET	`/api/loans?active=`	loan history / open loans
POST	`/api/loans`	check out (409 if unavailable)
POST	`/api/loans/{id}/return`	check in



Swagger /docs: POST /api/loans → 201 Created

SQLAlchemy 2 models · Pydantic validation · a partial unique index guarantees one open loan per item, even under concurrent checkouts

Tests: the first quality gate

backend — ruff + pytest

```
$ ruff check . && ruff format --check .
All checks passed!
22 files already formatted

$ pytest -v
============================================= test session starts ==============================================
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.7.0 -- ~/Code/gearloop/backend/.venv/bin/python3
rootdir: ~/Code/gearloop/backend
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collecting ... collected 21 items

tests/test_items.py::test_create_and_get_item PASSED                                                  [  4%]
tests/test_items.py::test_duplicate_asset_tag_is_rejected PASSED                                      [  9%]
tests/test_items.py::test_invalid_payload_is_rejected PASSED                                          [ 14%]
tests/test_items.py::test_update_condition_moves_item_to_maintenance PASSED                           [ 19%]
tests/test_items.py::test_list_filters_by_status_and_search PASSED                                    [ 23%]
tests/test_items.py::test_delete_item PASSED                                                          [ 28%]
tests/test_items.py::test_missing_item_returns_404 PASSED                                             [ 33%]
tests/test_loans.py::test_checkout_marks_item_on_loan PASSED                                          [ 38%]
tests/test_loans.py::test_cannot_double_checkout PASSED                                               [ 42%]
tests/test_loans.py::test_cannot_checkout_item_in_maintenance PASSED                                  [ 47%]
tests/test_loans.py::test_loan_length_is_capped PASSED                                                [ 52%]
tests/test_loans.py::test_return_frees_the_item PASSED                                                [ 57%]
tests/test_loans.py::test_cannot_delete_item_on_loan PASSED                                           [ 61%]
tests/test_loans.py::test_overdue_loans_show_in_stats PASSED                                          [ 66%]
tests/test_loans.py::test_active_filter_on_loan_list PASSED                                           [ 71%]
tests/test_loans.py::test_stats_on_empty_inventory PASSED                                             [ 76%]
tests/test_meta.py::test_health_is_up PASSED                                                          [ 80%]
tests/test_meta.py::test_root_identifies_service PASSED                                               [ 85%]
tests/test_meta.py::test_ready_when_schema_exists PASSED                                              [ 90%]
tests/test_meta.py::test_metrics_exposes_http_and_inventory_gauges PASSED                             [ 95%]
tests/test_seed.py::test_seed_is_idempotent PASSED                                                    [100%]

============================================== 21 passed in 0.24s ==============================================
```

21
backend tests (pytest)

5
frontend tests (vitest)

- Tests run on in-memory SQLite: never the real database
- CI also migrates a real PostgreSQL 17 and runs alembic check to catch model/migration drift
- ruff lint + format check before anything is built
- A failing test stops the pipeline: no image is pushed

Docker: one command, four services

docker compose up --build

```
$ docker compose up --build -d
Container gearloop-postgres-1 Started
Container gearloop-postgres-1 Healthy
Container gearloop-migrate-1 Started
Container gearloop-migrate-1 Exited
Container gearloop-backend-1 Started
Container gearloop-backend-1 Healthy
Container gearloop-frontend-1 Started

$ docker compose ps -a
SERVICE    IMAGE                    STATUS                                  PORTS
backend    gearloop-backend:local   Up 5 seconds (healthy)                  0.0.0.0:8000->8000/tcp
frontend   gearloop-frontend:local  Up Less than a second (health: starting)  0.0.0.0:3000->8080/tcp
migrate    gearloop-backend:local   Exited (0) 6 seconds ago
postgres   postgres:17-alpine       Up 13 seconds (healthy)                 0.0.0.0:5432->5432/tcp

$ docker compose logs migrate --no-log-prefix
INFO  [alembic.runtime.migration] Running upgrade  -> 0001_create_items, create items table
INFO  [alembic.runtime.migration] Running upgrade 0001_create_items -> 0002_create_loans, create loans table
seeded 10 demo items

$ docker compose exec backend id; docker compose exec frontend id
uid=10001(gearloop) gid=10001(gearloop) groups=10001(gearloop)
uid=101(nginx) gid=101(nginx) groups=101(nginx)
```

Startup order is enforced: postgres healthy → migrate exits 0 → backend healthy → frontend

Multi-stage builds: Node only in the frontend build stage; backend venv built separately

Non-root: backend uid 10001, unprivileged nginx uid 101

No pip at runtime: removes its vendored CVEs (next slides)

CI/CD pipeline (GitHub Actions)



Commit-SHA tags

every image traces back to one commit; never :latest

Actions pinned to SHAs

a moved or hijacked tag cannot change what runs

PRs build, main ships

images are pushed only from main

Pull-based deploy

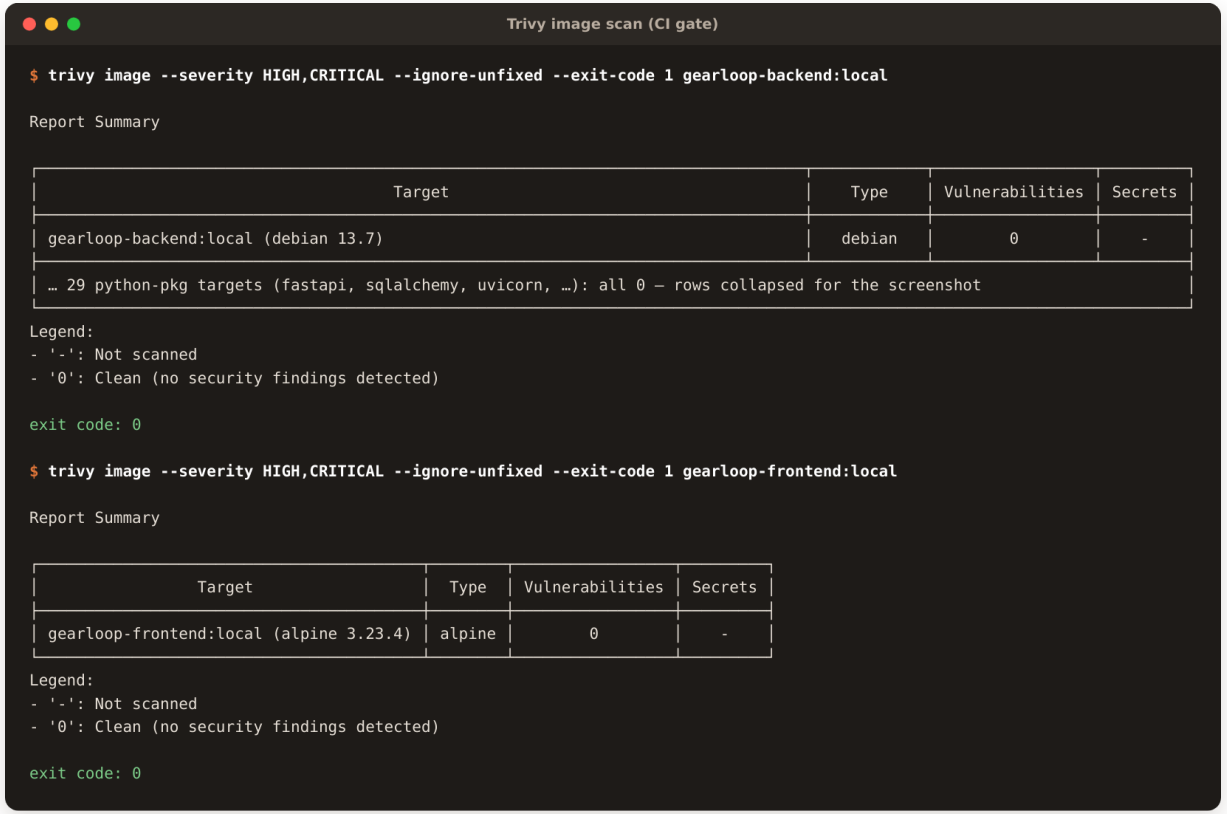
CI never holds cluster credentials; Argo CD pulls from Git

DevSecOps: the Trivy gate earned its place

4 HIGH → 0

fixable HIGH / CRITICAL CVEs in the backend image

- First backend build failed the gate: urllib3 (×2), msgpack, setuptools
- None were GearLoop dependencies: all were copies vendored inside pip itself
- The app never runs pip, so the fix was to remove pip from the runtime image, not to ignore the findings
- CI scans twice: full SARIF report to GitHub Security, then a blocking HIGH/CRITICAL gate



Terraform: the cluster setup as code

No cloud account needed: the kubernetes and helm providers manage the local cluster, so plan, apply and destroy really run.

kubernetes_namespace_v1
gearloop (PSA restricted) + monitoring

↓

quota + limit range
caps what the namespace can request

↓

helm_release.monitoring
kube-prometheus-stack 92.9.0 (CRDs first)

↓

helm_release.gearloop
../helm/gearloop · atomic, waits for Ready

6
resources

~40 s
apply on minikube

0
left after destroy

terraform apply

```
$ terraform apply tf.plan
kubernetes_namespace_v1.monitoring[0]: Creating...
kubernetes_namespace_v1.gearloop: Creating...
kubernetes_namespace_v1.gearloop: Creation complete after 0s [id=gearloop]
kubernetes_namespace_v1.monitoring[0]: Creation complete after 0s [id=monitoring]
kubernetes_resource_quota_v1.gearloop: Creating...
kubernetes_limit_range_v1.gearloop: Creating...
kubernetes_limit_range_v1.gearloop: Creation complete after 0s [id=gearloop/gearloop-defaults]
kubernetes_resource_quota_v1.gearloop: Creation complete after 0s [id=gearloop/gearloop-quota]
helm_release.monitoring[0]: Creating...
helm_release.monitoring[0]: Still creating... [00m20s elapsed]
helm_release.monitoring[0]: Creation complete after 25s [id=monitoring]
helm_release.gearloop: Creating...
helm_release.gearloop: Creation complete after 15s [id=gearloop]

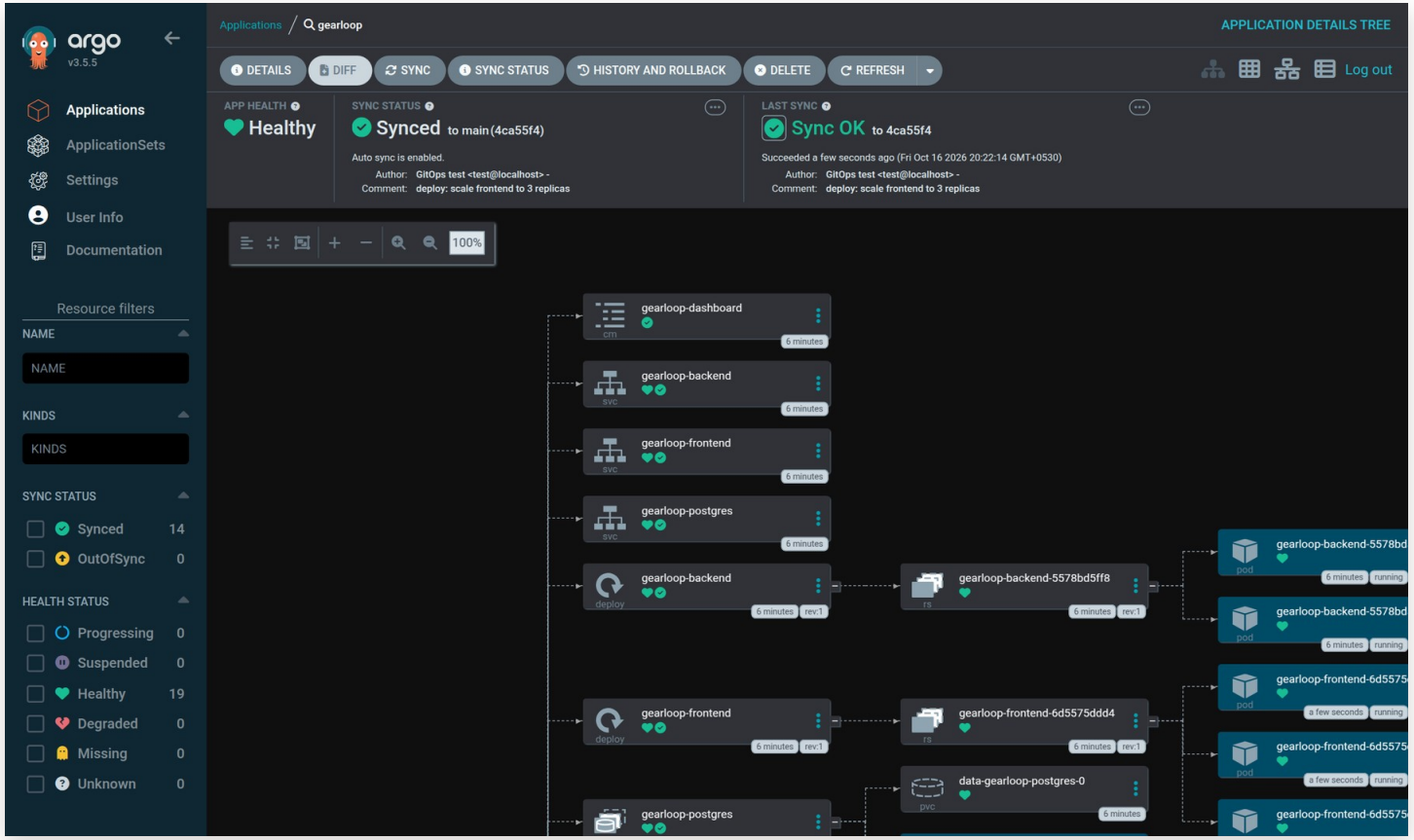
Apply complete! Resources: 6 added, 0 changed, 0 destroyed.

$ terraform output
app_url = "http://gearloop.local/"
grafana_url = "http://grafana.gearloop.local/"
namespace = "gearloop"
release = "gearloop rev 1 (deployed)"

$ helm list -A
NAME         NAMESPACE    REVISION  STATUS     CHART
gearloop     gearloop     1         deployed   gearloop-0.1.0
monitoring   monitoring   1         deployed   kube-prometheus-stack-92.9.0
```

Real run on minikube; destroy then removed both namespaces and the Postgres volume

GitOps with Argo CD



Synced / Healthy; tested against a local Git server before the GitHub push

18 s

from git push to synced pods

2 s

self-heal reverts a manual kubectl change

0

secrets in Git (DB Secret comes from Terraform)

CI commits the new image tag to values-gitops.yaml; Argo CD (auto-sync, prune, self-heal) pulls it. GitHub never needs access to the cluster.

Kubernetes + Helm

kubectl — namespace gearloop

```
$ kubectl get pods,svc,ingress,hpa,pdb -n gearloop
NAME                                    READY   STATUS      RESTARTS   AGE
pod/gearloop-backend-74fb869b7-7mfvr    1/1     Running     0          19m
pod/gearloop-backend-74fb869b7-9c7nh    1/1     Running     0          2m51s
pod/gearloop-backend-74fb869b7-hn2n7    1/1     Running     0          2m21s
pod/gearloop-backend-74fb869b7-rp9jt    1/1     Running     0          2m51s
pod/gearloop-backend-74fb869b7-s7hhr    1/1     Running     0          19m
pod/gearloop-backend-74fb869b7-zhpjz    1/1     Running     0          2m21s
pod/gearloop-frontend-6d5575ddd4-kqp6z  1/1     Running     0          19m
pod/gearloop-frontend-6d5575ddd4-mtcfr  1/1     Running     0          19m
pod/gearloop-postgres-0                 1/1     Running     0          19m

NAME                       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/gearloop-backend   ClusterIP   10.110.85.129   <none>        8000/TCP   19m
service/gearloop-frontend  ClusterIP   10.96.133.239   <none>        80/TCP     19m
service/gearloop-postgres  ClusterIP   None            <none>        5432/TCP   19m

NAME                                CLASS   HOSTS            ADDRESS        PORTS   AGE
ingress.networking.k8s.io/gearloop  nginx   gearloop.local   192.168.49.2   80      19m

NAME                                                  REFERENCE                     TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/gearloop-backend  Deployment/gearloop-backend   cpu: 170%/60%   2         6         6          19m

NAME                                      MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
poddisruptionbudget.policy/gearloop-backend   1               N/A               5                     19m
poddisruptionbudget.policy/gearloop-frontend  1               N/A               1                     19m
```

helm history: rev 1, installed by terraform apply **helm test:** **Succeeded**

Deployments
≥2 replicas, zero-downtime rolling updates

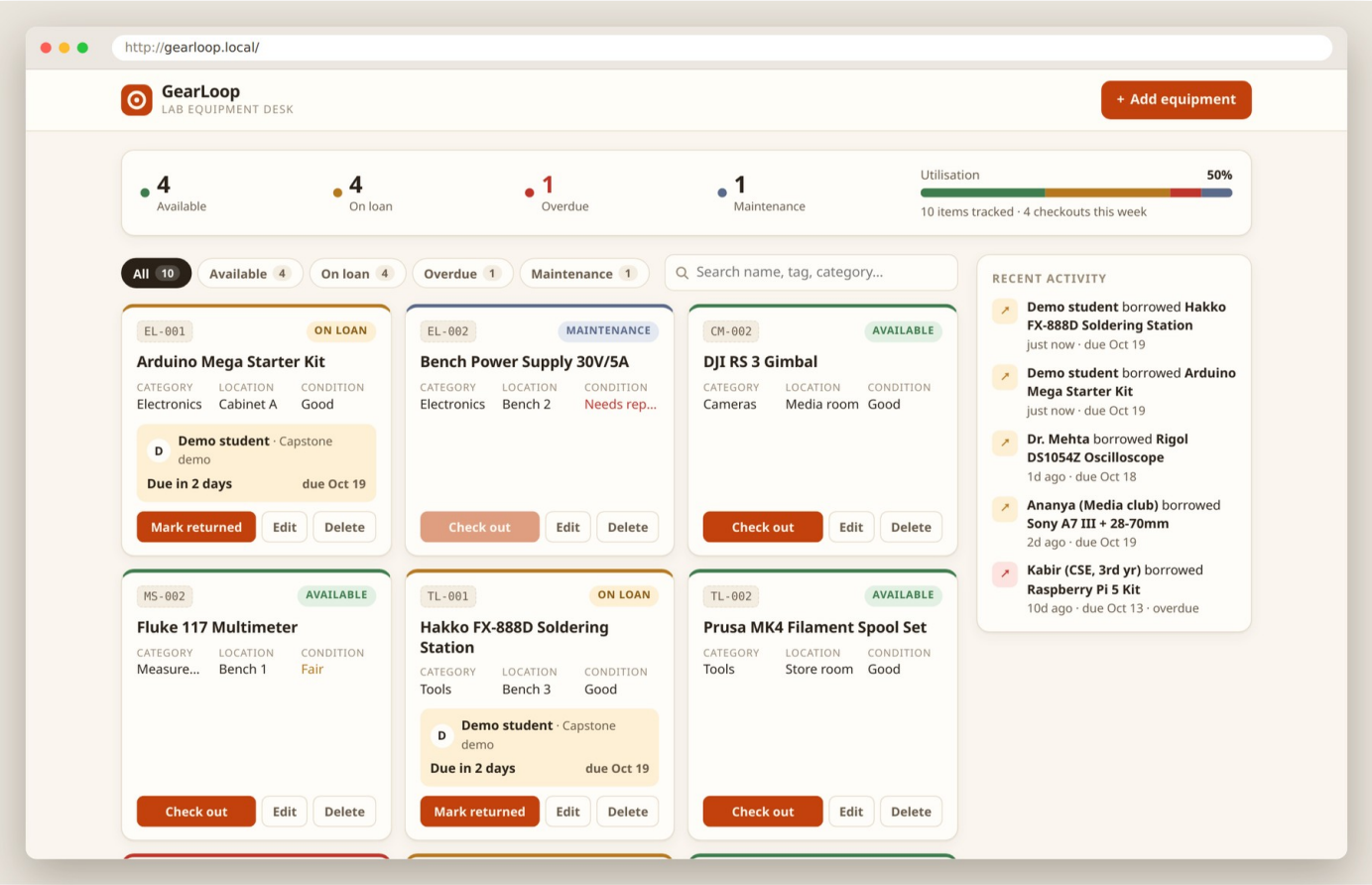
Postgres StatefulSet
PVC, password kept across upgrades

Migrations in an initContainer
Postgres advisory lock, so replicas never race

Startup, readiness, liveness
/ready gates traffic on the DB

Locked down
PSA restricted · read-only FS · NetworkPolicy · PDBs

Ingress: one origin for the browser



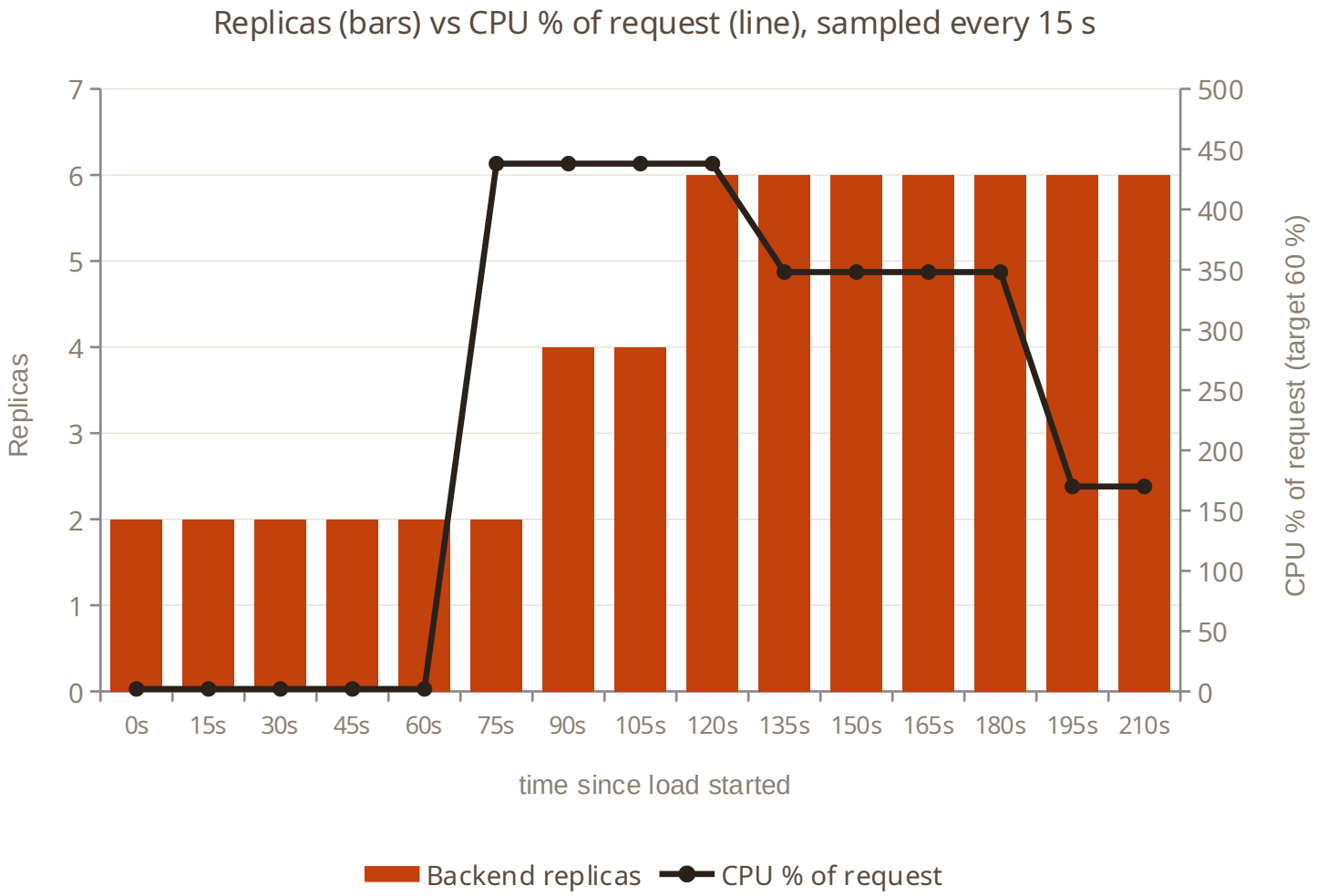
Path	Routed to
`/`	frontend (nginx)
`/api`	backend (FastAPI)
`/docs`	backend (Swagger)
`/openapi.json`	backend

No CORS and no backend hostname in the JavaScript bundle. The Ingress is only configuration; the ingress-nginx controller does the routing.

Bug found while capturing this deck

After a node restart, nginx exited with "host not found in upstream" before CoreDNS was ready. It now resolves the Service per request and stays up.

Autoscaling under load (real run)



2 → 6

backend replicas

438 %

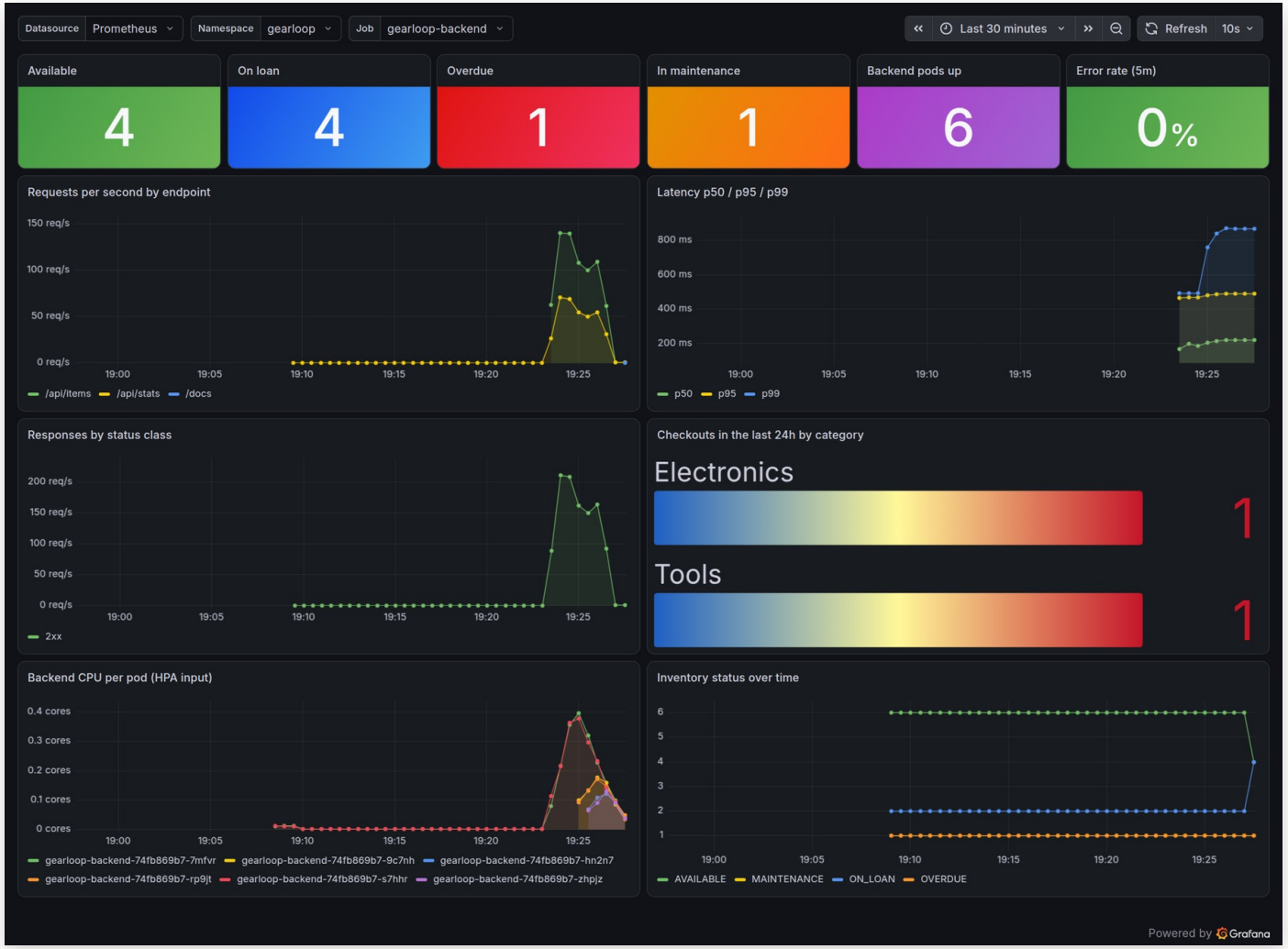
CPU of request at peak (target 60 %)

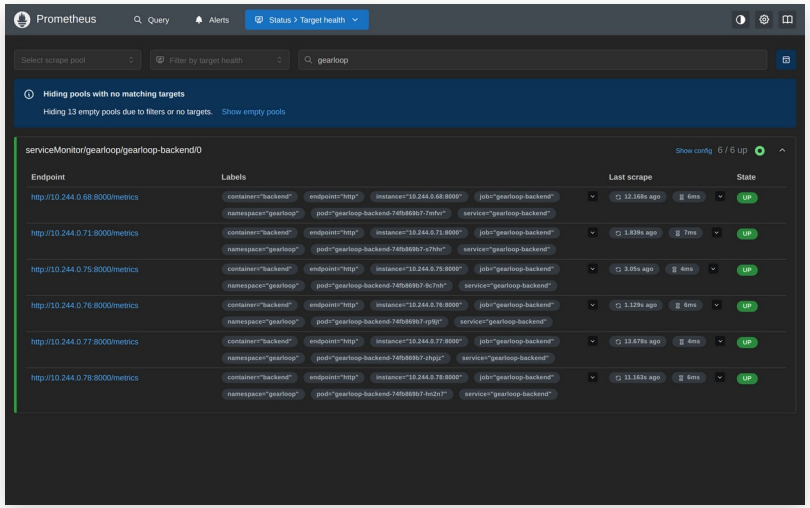
~45 s

from first scale-up to six pods

scripts/load-test.sh runs 30 in-cluster workers. The HPA needs CPU requests on the pods and a metrics source (metrics-server).

Observability: Prometheus + Grafana





Prometheus: every backend pod scraped and UP

Dashboard as code: shipped in the chart and auto-imported

Business metrics: items by status and checkouts per category, read from the DB at scrape time

4 alert rules: backend down, 5xx > 5 %, p95 > 500 ms, overdue gear

What testing on a real cluster caught

Helm install deadlock

Migrations ran in a post-install hook, but Helm waits for Ready pods first, and pods were not Ready without tables.

Fix initContainer + Postgres advisory lock

Frontend crash loop

After a node restart nginx resolved the backend before DNS was up and exited.

Fix Resolve per request with the Service FQDN

4 HIGH CVEs

Trivy blocked the backend image: pip's vendored urllib3, msgpack, setuptools.

Fix Remove pip from the runtime image

Wrong checkout counts

Per-pod counters lose the first event and disagree across replicas.

Fix Gauge read from the DB at scrape time

Troubleshooting lab: 5 broken manifests

troubleshooting lab — 5 broken manifests

```
$ kubectl apply -f troubleshooting/
$ kubectl get pods -n gearloop -l "app in (gearloop-canary,gearloop-staging-api,gearloop-report-worker)"
NAME                                    READY   STATUS         RESTARTS      AGE
gearloop-canary-5965786fd5-jwbrl        0/1     ErrImagePull   0             12s
gearloop-report-worker-69cffc657b-4szkz 0/1     OOMKilled      1 (11s ago)   12s
gearloop-staging-api-6f867c68d4-wc2n9   0/1     Running        0             12s

# 1  ImagePullBackOff
$ kubectl describe pod -l app=gearloop-canary -n gearloop | grep -m1 "Failed to pull"
  Failed to pull image "ghcr.io/aastik-gg/gearloop-backend:v9.9.9-typo"

# 2  Service with no endpoints (selector component=api, pods are component=backend)
$ kubectl get endpointslices -n gearloop -l kubernetes.io/service-name=gearloop-api-internal
NAME                        ADDRESSTYPE   PORTS     ENDPOINTS   AGE
gearloop-api-internal-ptlxw IPv4          <unset>   <unset>     12s

# 3  Running but never Ready (wrong DB password)
$ kubectl get events -n gearloop --field-selector reason=Unhealthy | grep staging
  pod/gearloop-staging-api-6f867c68d4-wc2n9 Readiness probe failed: HTTP probe failed with statuscode: 503
$ kubectl exec gearloop-staging-api-6f867c68d4-wc2n9 -n gearloop -- python -c "urlopen('http://127.0.0.1:8000/ready')"
  503 {"status":"NOT_READY","reason":"OperationalError"}
$ kubectl get pod gearloop-staging-api-6f867c68d4-wc2n9 -n gearloop -o jsonpath='{..ready}'
  false

# 4  OOMKilled
$ kubectl get pod -l app=gearloop-report-worker -n gearloop -o jsonpath='{..lastState.terminated}'
{'exitCode': 137, 'reason': 'OOMKilled'}

# 5  Ingress -> wrong Service port (8080 instead of 8000)
$ curl -s -o /dev/null -w "%{http_code}" http://beta.gearloop.local/api/stats
503
```

Symptom	Root cause
ImagePullBackOff	tag that was never pushed
No endpoints	Service selector matches no pods
Running, not Ready	wrong DB password → /ready 503
OOMKilled (137)	memory limit below real usage
Ingress 503	points at Service port 8080, not 8000

Each scenario has a guided diagnosis and fix in troubleshooting/README.md. All five were applied to the cluster and produce exactly the documented symptom.

Key learnings

Test on a real cluster

Unit tests passed for all four bugs on the previous slide; only real installs, restarts and scans exposed them.

Readiness is not liveness

A bad DB password should stop traffic (/ready), not restart-loop the pod (/health).

Pull beats push for deploys

With GitOps, Git is the record of what runs, and CI needs no cluster credentials.

Fix CVEs at the source

Removing pip from the image beat ignoring four HIGH findings.

Own what you create

Terraform owning the namespaces made destroy leave nothing behind.

Measure from the source of truth

DB-backed gauges agree across replicas; per-pod counters did not.

What is verified, and what is next

	Area	Evidence
Done	**App, tests, migrations**	21 pytest + 5 vitest; migrations on PostgreSQL 17
Done	**UI**	every dialog clicked through on the live stack; mobile at 390px
Done	**Docker + Trivy**	compose brings up 4 services; both images non-root and scan clean
Done	**Kubernetes + Helm**	installed by Terraform; upgrades + rolling restart tested; helm test passing
Done	**Ingress · HPA · monitoring**	routing verified; HPA 2 → 6; targets UP; dashboard populated
Done	**Troubleshooting lab**	all 5 scenarios reproduce their symptom
Done	**Terraform**	plan / apply / destroy on minikube; destroy leaves nothing behind
Done	**Argo CD (GitOps)**	local Git server: sync in 18 s, self-heal in 2 s; GitHub-backed after the push
Next	**GitHub Actions**	actionlint clean; runs once the repo is pushed to GitHub

Live demo

Code → CI/CD → Terraform → Argo CD → Kubernetes → Prometheus → Grafana

Thank you

1. Codebase: folder structure and key files
2. Push a change; the pipeline runs end to end
3. Terraform plan / state on the local cluster
4. Argo CD: synced app, GitOps flow, self-heal
5. Kubernetes: pods, services, ingress, the app
6. Prometheus targets UP
7. Grafana live while the HPA scales out